Introducción a la Dinámica de Fluidos Computacional y Fenómenos de Transporte Computacional

INFORME TÉCNICO - EXAMEN PARCIAL
11 de junio de 2026

Andrea Grimaldi

SiCaN LAB
FACULTAD de INGENIERIA del EJERCITO

Laboratorio de Simulación y Cálculo Numérico – SiCaNLab
Facultad de Ingeniería del Ejército, Universidad de la Defensa Nacional
Av. Cabildo 15 (C1426AAA) C.A.B.A. – República Argentina
https://www.fie.undef.edu.ar/

---

1 Introduction

The object of study is a metallic plate of dimensions $0.1 \times 0.1 \times 0.01$ m, made up of two regions with different thermal properties. Sector A is a square of $0.04 \times 0.04 \times 0.01$ m with thermal conductivity $k_A = 100\,\mathrm{W\,m^{-1}\,K^{-1}}$ and internal heat generation $\dot{q} = 5.0 \times 10^5\,\mathrm{W\,m^{-3}}$. Sector B is the remaining part, with $k_B = 550\,\mathrm{W\,m^{-1}\,K^{-1}}$ and no heat generation. The main goal is to obtain the temperature field $T(x, y)$ in the study domain, using the Finite Volume Method (FVM) for two geometric configurations of the plate, later defined.

1.1 Governing equation

The starting point is the general transport equation for a generic scalar ϕ [1], which expresses the balance of that quantity over a fixed control volume. In the conservative form it contains four terms,

$$\underbrace{\frac{\partial(\rho\,\phi)}{\partial t}}_{\text{transient}} + \underbrace{\nabla\cdot(\rho\,\mathbf{u}\,\phi)}_{\text{convection}} = \underbrace{\nabla\cdot(\Gamma\,\nabla\phi)}_{\text{diffusion}} + \underbrace{S_\phi}_{\text{source}}\,, \tag{1}$$

where ρ is the density, $\mathbf{u}$ the velocity field, Γ the diffusion coefficient, and S_ϕ the volumetric source. For our case (heat conduction) the transported scalar is the temperature, $\phi = T$, the diffusion coefficient is the thermal conductivity, $\Gamma = k$, the source is the heat generation, $S_\phi = \dot{q}$, and $\rho\,\phi$ becomes the stored energy $\rho c T$. Three simplifications reduce Eq. (1) to the form solved here.

First, the plate is solid, so there is no flow: $\mathbf{u} = \mathbf{0}$ and the convection term vanishes. The balance becomes pure diffusion with a source,

$$\rho c \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + \dot{q}. \tag{2}$$

Second, the analysis is steady state. The temperature does not change in time, so $\partial T/\partial t = 0$ and the transient term vanishes:

$$\nabla \cdot (k \nabla T) + \dot{q} = 0. \tag{3}$$

Third, the front and back faces are adiabatic, $\partial T/\partial z = 0$, and neither the material properties nor the heat generation vary across the z-axis. The 3D problem therefore collapses in a two dimensional problem. The height h of the plate then enters only as a constant geometric factor in the face areas and control-volume sizes. Expanding the divergence operator we obtain the equation to solve:

$$\frac{\partial}{\partial x}\left(k \frac{\partial T}{\partial x}\right) + \frac{\partial}{\partial y}\left(k \frac{\partial T}{\partial y}\right) + \dot{q} = 0. \tag{4}$$

Because the equation is steady, all the nodes are coupled at once and the discrete system is solved with an iterative method, rather than advanced step by step in time.

1.2 Boundary conditions

The four lateral surfaces and the front and back faces impose three types of boundary condition. The south surface has a Dirichlet condition (fixed temperature on the surface), with prescribed temperature $T = 30\,^{\circ}\mathrm{C}$. The north surface and the front and back faces have a Neumann (adiabatic) condition, so the heat flux through those surfaces is zero. The west and east surfaces have a Robin (convective) condition, exchanging heat with a fluid at $T_\infty = 15\,^{\circ}\mathrm{C}$ through a convective coefficient $h_b = 200\,\mathrm{W\,m^{-2}\,K^{-1}}$.

Surface	**Type**	**Condition**
North	Neumann (adiabatic)	$\partial T/\partial y = 0$
South	Dirichlet	$T = 30\,^{\circ}\mathrm{C}$
West / East	Robin (convective)	$T_\infty = 15\,^{\circ}\mathrm{C}, \quad h_b = 200\,\mathrm{W\,m^{-2}\,K^{-1}}$
Front / Back	Neumann (adiabatic)	$\partial T/\partial z = 0$

Table 1: Boundary conditions imposed on the six faces of the plate.

1.3 The Finite Volume Method

The Finite Volume Method (FVM) solves the transport equation by dividing the domain into a set of control volumes, each with a node at its center (the node is in fact called centroid). The equation is then integrated over every control volume, which turns the differential balance into an algebraic statement: the net heat entering a cell through its faces, plus any heat generated inside it, must be zero at steady state.

The discretization follows from integrating the steady diffusion equation, Eq. (4), over a control volume V around node P:

$$\int_V \nabla \cdot (k\,\nabla T)\; dV + \int_V \dot{q}\, dV = 0. \tag{5}$$

Applying the divergence (Gauss) theorem, the volume integral of the divergence becomes a sum of fluxes over the faces of the cell:

$$\oint_{\partial V} (k\,\nabla T) \cdot \mathbf{n}\, dA + \dot{q}\, V = 0, \tag{6}$$

where $\mathbf{n}$ is the outward face normal. For the present two-dimensional problem this surface integral reduces to a sum over the four faces (east, west, north, south) of the cell. The same face-flux result can be obtained without using the divergence theorem, by expanding the divergence into Cartesian derivatives, Eq. (4), integrating each term over the cell, and applying the fundamental theorem of calculus one direction at a time. For the east and west faces,

$$\int_{y_s}^{y_n}\int_{x_w}^{x_e} \frac{\partial}{\partial x}\left(k\,\frac{\partial T}{\partial x}\right) h\, dx\, dy = \left[\left(k\,\frac{\partial T}{\partial x}\right)\Big|_e - \left(k\,\frac{\partial T}{\partial x}\right)\Big|_w\right] \Delta y\, h, \tag{7}$$

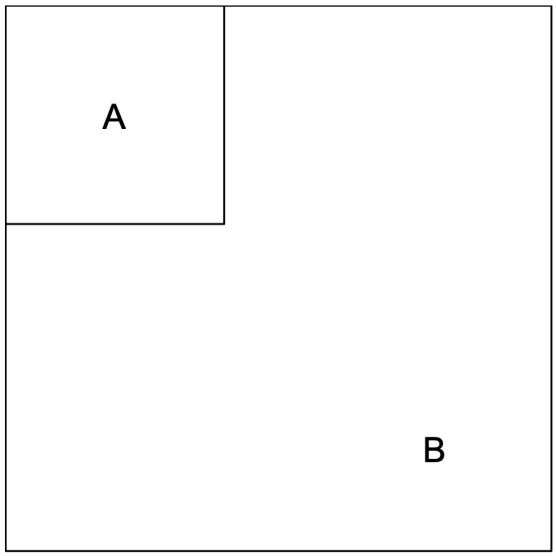
and analogously in y for the north and south faces. The divergence theorem is the general, coordinate-free form of this step; however since we are considering rectangular cells aligned with the x and y axis we can avoid using the said theorem. Either way, the problem is reduced to evaluating the gradient at each face. Since the discrete temperatures are known only at the nodes, each face gradient is approximated by a two-point difference; for the east face, for example, $\partial T/\partial x|_e \approx (T_E - T_P)/\delta x_e$. Substituting these approximations for all faces and grouping the temperatures leaves us one algebraic equation per node,

$$a_P\, T_P = a_E\, T_E + a_W\, T_W + a_N\, T_N + a_S\, T_S + b, \tag{8}$$

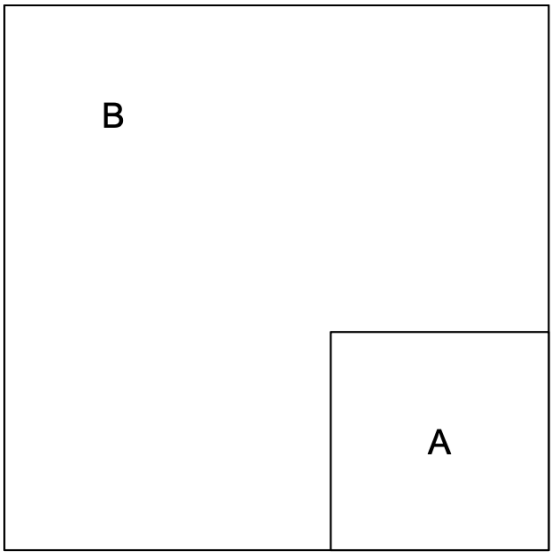
where the neighbour coefficients (a_E, a_W, a_N, a_S) collect the face conductances, b holds the source and boundary contributions, and $a_P = a_E + a_W + a_N + a_S$. Assembling Eq. (8) for all nodes gives a coupled linear system that is solved for the temperature field. Because it is built directly from face fluxes, the method conserves energy exactly in each cell and over the whole domain.

1.4 Configurations and objective

Two configurations of the plate are studied. Configuration 1 places sector A at the top-left of the domain; configuration 2 places it at the bottom-right. Figures 1a and 1b show the two cases.



(a) Configuration 1. Sector A at the top-left.



(b) Configuration 2. Sector A at the bottom-right.

Figure 1: The two geometric configurations of the plate.

For each configuration the FVM solution is used to produce two results: a filled contour plot of the temperature field $T(x, y)$, and the temperature profile along the diagonal that crosses sector A (which is the same for both configurations, specifically the one that goes from top left to bottom right).

2 Development

2.1 Algebraic development

The steady diffusion equation with a source, Eq. (3), is solved by the Finite Volume Method on a structured orthogonal mesh [1]. The domain is divided into $n_x \times n_y$ rectangular control volumes, each holding one node at its centroid. Integrating the equation over a control volume and reducing the volume integral to a sum of face fluxes, through Eq. (6) or Eq. (7), we obtain one algebraic equation per node,

$$a_P\, T_P = a_E\, T_E + a_W\, T_W + a_N\, T_N + a_S\, T_S + b, \tag{9}$$

where the neighbour coefficients collect the face conductances and b holds the source and boundary contributions.

The four interior neighbour coefficients are

$$a_E = \frac{k_e\, \Delta y\, h}{\delta x_e}, \qquad a_W = \frac{k_w\, \Delta y\, h}{\delta x_w}, \qquad a_N = \frac{k_n\, \Delta x\, h}{\delta y_n}, \qquad a_S = \frac{k_s\, \Delta x\, h}{\delta y_s}, \tag{10}$$

where k_e, k_w, k_n, k_s are the conductivities evaluated at the east, west, north and south faces; δx and δy are the distances between adjacent nodes; and h is the plate height. For a uniform mesh the node–node distances are equal to the cell sizes, $\delta x_e = \Delta x$ and $\delta y_n = \Delta y$. The source term is non-zero only in the cells that belong to sector A,

$$b = \begin{cases} \dot{q}\,\Delta x\,\Delta y\,h & \text{if cell } P \text{ lies in sector A,} \\ 0 & \text{otherwise.} \end{cases} \tag{11}$$

The central coefficient is the sum of the neighbour coefficients, $a_P = a_E + a_W + a_N + a_S$.

2.2 Geometry and mesh

The plate is made of two materials, and this imposes a constraint on the mesh: a single control volume cannot contain part of sector A and part of sector B at the same time. The reason is that the face conductivity used at a material change, introduced in Eq. (14) below, is only valid when the jump in conductivity falls exactly on a cell face. If the A–B boundary passed through the interior of a cell, that cell would have an undefined conductivity and the heat flux at the interface would not be represented correctly. With a plate side of 0.1 m and a sector side of 0.04 m, the interface lands on a cell face only when n_x and n_y are multiples of five. This condition is enforced in the code with assertions on `nx` and `ny`.

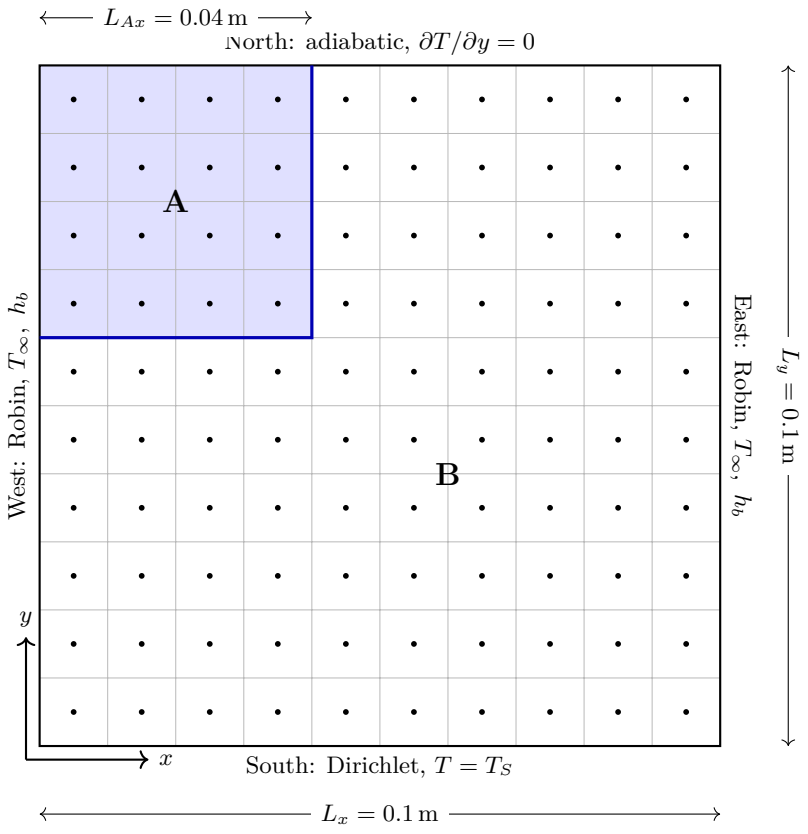


Figure 2: Configuration 1 on a uniform 10×10 mesh. Sector A (0.04 m side, shaded) occupies the top-left 4×4 block of control volumes; sector B fills the remainder. The material interface (thick blue line) coincides with cell faces, which is the condition that makes the harmonic-mean face conductivity valid. The boundary conditions on the four lateral surfaces are also indicated.

The nodes are placed at the cell centres,

$$x_{P_i} = \left(i + \tfrac{1}{2}\right)\Delta x, \qquad y_{P_j} = \left(j + \tfrac{1}{2}\right)\Delta y, \tag{12}$$

with $i = 0, \ldots, n_x - 1$ and $j = 0, \ldots, n_y - 1$. The two configurations differ only in the position of sector A. A single flag `config` selects the case (allowing the code to be used for both configurations without any other changes), and a helper function decides whether a cell (i, j) belongs to sector A,

$$\text{cell } (i,j) \in A \iff \begin{cases} i < n_{Ax} \ \wedge \ j \geq n_y - n_{Ay} & \text{(Configuration 1)}, \\ i \geq n_x - n_{Ax} \ \wedge \ j < n_{Ay} & \text{(Configuration 2)}, \end{cases} \tag{13}$$

where $n_{Ax} = L_{Ax}/\Delta x$ and $n_{Ay} = L_{Ay}/\Delta y$ are the number of cells spanned by sector A in each direction. Because only this membership test changes between the two cases, the discretization and the solver are identical for both.

2.3 Material interface

Each cell is assigned (for its faces) the conductivity of the material it occupies: k_A inside sector A and k_B in sector B. The conductivity at a face between two cells P and E is taken as the harmonic mean of the two cell values [1],

$$k_f = \frac{2\,k_P\,k_E}{k_P + k_E}. \tag{14}$$

This expression is the special case of the general series-resistance, which would also work in the case where the distance from the face to the two nodes is not the same. In this case, however, the distance is, by construction, the same, as it can be deduced from Eq. (12). It is the correct choice across a material change because it enforces continuity of the heat flux through the interface; an arithmetic mean would not, especially if one of the two materials is an insulator ($k_P \to 0$). At a face between sector A and sector B it returns

$$k_f = \frac{2\,(100)(550)}{100 + 550} \approx 169.2\,\mathrm{W\,m^{-1}\,K^{-1}}, \tag{15}$$

while at a face inside a single material it reduces to that material's conductivity.

2.4 Boundary conditions

The three boundary types of Table 1 are imposed by modifying the coefficients of the boundary cells. For an interior face the neighbour coefficient links two nodes, while for a boundary face it must instead link the boundary node to the given external condition.

South face (Dirichlet): The south surface is held at a fixed temperature T_S. The distance from a south-row node to the surface is half a cell, $\Delta y/2$, so the boundary conductance is

$$a_b = \frac{k_P\,\Delta x\,h}{\Delta y/2}. \tag{16}$$

This term is added to the central coefficient, $a_P \mathrel{+}= a_b$, and the prescribed temperature enters the source as $b \mathrel{+}= a_b\,T_S$. The factor of two in the denominator reflects that the node sits half a cell away from the surface, not a full cell.

West and east faces (Robin): These surfaces exchange heat with a fluid at T_∞ through a convective coefficient h_b. Consider the west face of a boundary cell with node P and surface temperature T_s (unknown). At steady state the same heat rate Q crosses the two stages in series. The conduction from the node to the surface, across the half-cell of width $\Delta x/2$, is

$$Q = k_P \frac{T_P - T_s}{\Delta x/2} \Delta y\, h, \tag{17}$$

and the convection from the surface to the fluid is

$$Q = h_b \,(T_s - T_\infty)\, \Delta y\, h. \tag{18}$$

Rearranging the terms we obtain,

$$T_P - T_s = Q \underbrace{\frac{\Delta x/2}{k_P\, A}}_{\alpha}, \qquad T_s - T_\infty = Q \underbrace{\frac{1}{h_b\, A}}_{\beta}. \tag{19}$$

The surface temperature T_s is unknown and must be removed. Adding the two equations cancels it,

$$T_P - T_\infty = Q\,(\alpha + \beta) = Q\left(\frac{\Delta x/2}{k_P\, A} + \frac{1}{h_b\, A}\right). \tag{20}$$

Solving for the heat rate,

$$Q = \frac{T_P - T_\infty}{\dfrac{\Delta x}{2\, k_P\, A} + \dfrac{1}{h_b\, A}} = \frac{A}{\dfrac{\Delta x}{2\, k_P} + \dfrac{1}{h_b}}\,(T_P - T_\infty) = a_{\text{rob}}\,(T_P - T_\infty), \tag{21}$$

where the effective conductance of the face is

$$a_{\text{rob}} = \frac{\Delta y\, h}{\dfrac{\Delta x}{2\, k_P} + \dfrac{1}{h_b}}. \tag{22}$$

As in the Dirichlet case, this is added to the central coefficient, $a_P \mathrel{+}= a_{\text{rob}}$, and the fluid temperature enters the source, $b \mathrel{+}= a_{\text{rob}}\, T_\infty$. Equation (22) reduces to a Dirichlet condition as $h_b \to \infty$ (the convective resistance vanishes and the surface is pinned to T_∞) and to an adiabatic condition as $h_b \to 0$ (the convective resistance grows without bound and $a_{\text{rob}} \to 0$).

North face (Neumann, adiabatic): An adiabatic surface carries no heat flux through it. The heat rate (flux over the area) that a neighbour coefficient represents is, for a north face,

$$Q_n = \left(k\,\frac{\partial T}{\partial y}\right)\Big|_n \Delta x\, h, \tag{23}$$

so the Neumann condition $\partial T/\partial y = 0$ on that face sets this rate to zero directly,

$$Q_n = 0. \tag{24}$$

The face gradient is approximated by a two-point difference between node P and its north neighbour N,

$$\left(k\,\frac{\partial T}{\partial y}\right)\Big|_n \approx k_n\,\frac{T_N - T_P}{\delta y_n}, \tag{25}$$

so the north-face rate of Eq. (23) becomes

$$Q_n \approx \frac{k_n\,\Delta x\,h}{\delta y_n}\,(T_N - T_P) = a_N\,(T_N - T_P), \qquad a_N \equiv \frac{k_n\,\Delta x\,h}{\delta y_n}. \tag{26}$$

Bringing together Eq. (24) and Eq. (26)

$$\left.\frac{\partial T}{\partial y}\right|_n = 0 \quad \Longrightarrow \quad a_N\,(T_N - T_P) = 0 \quad \forall\,(T_N - T_P) \quad \Longrightarrow \quad a_N = 0. \tag{27}$$

Keeping it simpler, however, we can just say that there is no node beyond the boundary to difference against, so the approximation of Eq. (25) is never formed and nothing is added to a_P (hence a_N is zero) or b: the term is simply absent from the balance, Eq. (9). This is why no special treatment is needed in the code. The neighbour coefficient of a boundary cell is initialised to zero and overwritten only for interior faces; for an adiabatic face it is left at zero. The same applies to the front and back faces, whose adiabatic condition $\partial T/\partial z = 0$ is what reduced the problem to two dimensions in the first place.

2.5 Linear solver

The assembled system, Eq. (9) written for all the nodes, is solved with the Gauss–Seidel iterative method [1]. Starting from an initial guess, each node is updated in turn using the most recent values of its neighbours,

$$T_P^{(k+1)} = \frac{a_E\,T_E + a_W\,T_W + a_N\,T_N + a_S\,T_S + b}{a_P}, \tag{28}$$

where the neighbours already visited in the current sweep use their updated values. Convergence of the iteration is guaranteed by the Scarborough criterion [2], which requires the matrix to be diagonally dominant,

$$\frac{|a_E| + |a_W| + |a_N| + |a_S|}{|a_P|} \le 1 \quad \text{for every node, with strict inequality for at least one node.} \tag{29}$$

This condition is satisfied here because the boundary terms of Eqs. (16) and (22) add to a_P without adding to any neighbour coefficient. The iteration is stopped when the change in the temperature field between two successive sweeps, measured by ℓ^2 norm [3], falls below a tolerance of 10^{-6} (or when the maximum number of iterations is reached).

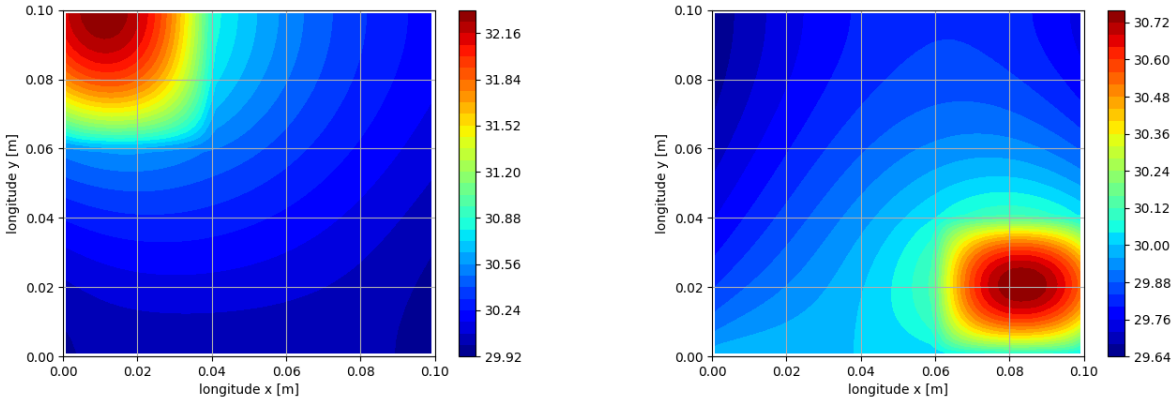
2.6 Results

The finite-volume problem was solved on a uniform mesh of $n_x = n_y = 50$ control volumes ($\Delta x = \Delta y = 0.002\,\mathrm{m}$, 2500 cells) for both configurations. The converged temperature fields are shown as filled contour plots in Figure 3, and the temperature profiles along the diagonal that crosses sector A are shown in Figure 4. The diagonal runs from the top-left corner of the domain to the bottom-right corner; the abscissa is the arc length measured from the top-left corner. Table 2 collects the global quantities of each case.

In Configuration 1 the temperature reaches its maximum of 32.30 °C inside sector A in the top-left region and decreases going towards the boundaries, with a minimum of 29.97 °C. Along the diagonal (Figure 4a) the temperature reaches its maximum in the first part (corresponding to sector A) and

	Configuration 1	Configuration 2
$T_{\max}$ [°C]	32.30	30.76
$T_{\min}$ [°C]	29.97	29.65
Q_{gen} [W]	8.00	8.00
Q_S [W]	1.84	2.04
Q_W [W]	3.15	2.95
Q_E [W]	3.01	3.01

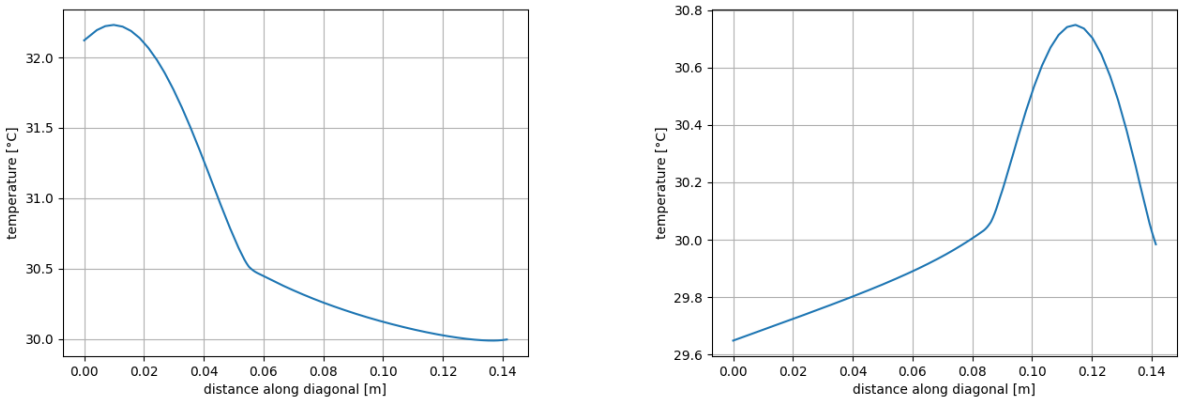
Table 2: Global results for the two configurations on the 50×50 mesh: extreme temperatures, generated power, and the dissipated power among the non-adiabatic surfaces (south, west, east).



(a) Configuration 1 (sector A at the top-left).

(b) Configuration 2 (sector A at the bottom-right).

Figure 3: Filled contour plots of the temperature field $T(x, y)$ in Celsius degrees. The hottest region coincides with sector A, where heat is generated; the south surface (held at T_S) and the west and east surfaces (cooled by convection) absorb the generated heat.



(a) Configuration 1.

(b) Configuration 2.

Figure 4: Temperature along the top-left to bottom-right diagonal, where the distance along diagonal equal to zero corresponds to the top-left. The diagonal crosses sector A near the start of the path in Configuration 1 and near the end in Configuration 2.

monotonically decreases to 30 °C (which is forced in the bottom-right of the plate by the Dirichlet condition on the south boundary).

In Configuration 2 the maximum temperature is 30.76 °C, again located inside sector A but now in the bottom-right region, and the minimum is 29.65 °C. The diagonal profile (Figure 4b) rises from the top-left corner, reaches its peak as it enters sector A near the bottom-right, and then drops over the short remaining segment to the corner, reaching 30 °C as in Configuration 1 (due to the Dirichlet condition).

For both configurations the generated power is $Q_{\text{gen}} = 8.0\,\text{W}$, which is also the value of the power leaving through the three non-adiabatic surfaces, as reported in Table 2; the verification of this balance is detailed in Section 2.7.

2.7 Verification of results

Two independent checks confirm that the discrete solution is consistent before the temperature fields are interpreted.

The first check is local. The residual of the discrete equation,

$$R_P = a_E\,T_E + a_W\,T_W + a_N\,T_N + a_S\,T_S + b - a_P\,T_P, \tag{30}$$

is evaluated at every node [2]. Its largest absolute value is below 10^{-4}, which confirms that Eq. (9) is satisfied at each cell to within the solver tolerance. A face-symmetry check is also performed: the east coefficient of a cell must equal the west coefficient of its right neighbour, and likewise for the north and south pair, which verifies that the harmonic-mean conductivity of Eq. (14) was assigned consistently from both sides of each face.

The second check is global and follows directly from energy conservation. At steady state the total heat generated inside sector A must equal the total heat leaving the plate through its non-adiabatic surfaces. The generated power is

$$Q_{\text{gen}} = \dot{q}\,L_{Ax}\,L_{Ay}\,h = (5\times10^{5})(0.04)(0.04)(0.01) = 8.0\,\text{W}. \tag{31}$$

The heat leaving through the south (Dirichlet) and the west and east (Robin) surfaces is recovered from the converged temperature field,

$$Q_S = \sum_i k_P\,\frac{T_{P,i} - T_S}{\Delta y/2}\,\Delta x\,h, \tag{32}$$

$$Q_W + Q_E = \sum_j \frac{T_{P,j} - T_\infty}{\dfrac{\Delta x}{2\,k_P} + \dfrac{1}{h_b}}\,\Delta y\,h. \tag{33}$$

The generated and dissipated power match in both configurations, within a relative error below 10^{-3}, confirming that the scheme conserves energy over the whole domain regardless of where sector A is placed.

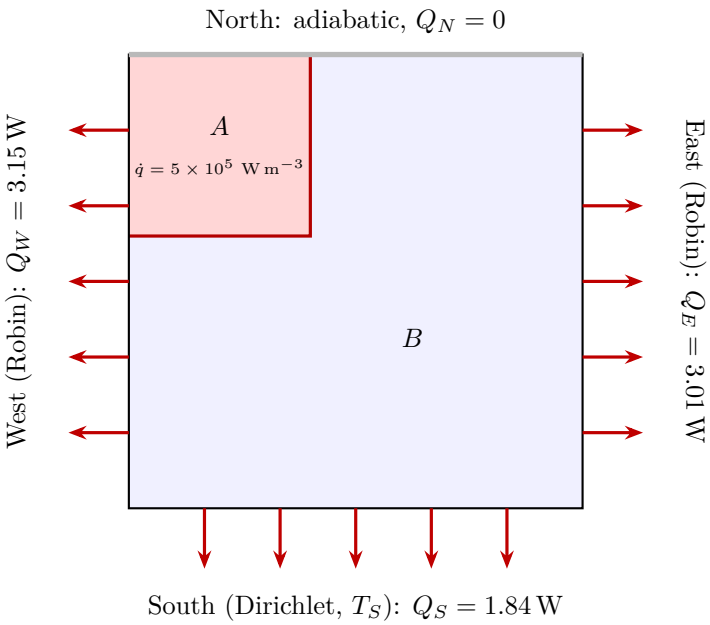


Figure 5: Geometric heat balance for Configuration 1. Power is generated inside sector A (red, top-left) at the volumetric rate $\dot{q}$ and leaves the plate through the three non-adiabatic surfaces.

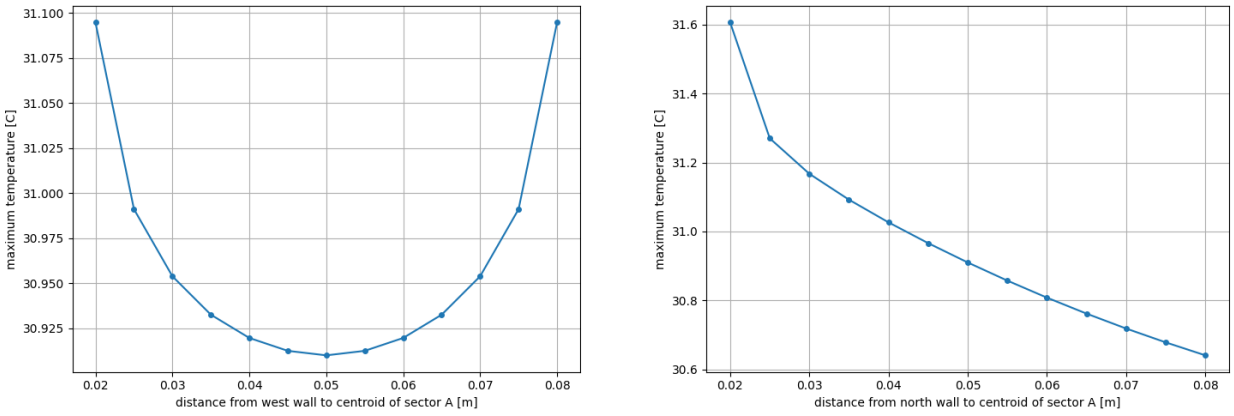
3 Conclusion

The two-dimensional steady heat-conduction problem in the metallic plate was solved by the Finite Volume Method for both geometric configurations of sector A. The temperature fields, the diagonal profiles, and the global energy balance were obtained on a uniform mesh and verified by an independent residual check and an energy-conservation check, both of which were satisfied.

The temperature fields show the expected qualitative behaviour: in both configurations the hottest region coincides with sector A, where heat is generated, and the temperature decreases towards the boundaries. The peak temperature differs between the two cases, and the following study clarifies why. The position of sector A was swept along each axis while the opposite coordinate was held fixed at the centre of the plate, and the maximum temperature was recorded at each position. The two sweeps reveal markedly different behaviour, and the difference is a direct consequence of the boundary conditions.

Moving sector A horizontally (at fixed $y = 0.05\,\mathrm{m}$) produces a symmetric curve with a minimum at the centre of the plate: the peak temperature is lowest, about 30.91 °C, when A is centred, and rises symmetrically to about 31.10 °C as A approaches either side wall. The symmetry is a consequence of the west and east faces carrying identical Robin conditions, so that moving A towards one wall or the other is physically equivalent; the plate is symmetric along the segment $x = 0.05\,\mathrm{m}$.

Moving sector A vertically (at fixed $x = 0.05\,\mathrm{m}$) produces instead a monotonic curve: the peak temperature falls from about 31.61 °C, when A is pressed against the north wall, to about 30.64 °C, when A sits against the south wall. There is no symmetry here, because the two horizontal faces carry different conditions: the north face is adiabatic and the south face has a Dirichlet boundary. When A approaches the adiabatic north wall, that wall blocks any outward flux and confines the generated heat, so the peak temperature rises sharply; when A approaches the south bound, the heat escapes easily and the peak is lowest. The rapid increase of the maximum temperature at the north wall marks the point where the last layer of sector B separating A from the wall disappears, removing the horizontal conduction region.



(a) Moving sector A along x-axis.

(b) Moving sector A along y-axis.

Figure 6: Maximum temperature in the plate as a function of the position of sector A, obtained by moving its centroid along one axis while the other coordinate is held at the centre of the plate.

Despite these variations in the temperature field, the total power generated and dissipated by the system is identical for every position of sector A, and in particular for the two configurations previously introduced (Figure 1), $Q_{\mathrm{gen}} = 8.0\,\mathrm{W}$. This is not a coincidence of the numerical solution but a direct consequence of the problem definition. The generated power depends only on the

volumetric heat-generation rate and on the volume of the generating region,

$$Q_{\text{gen}} = \dot{q}\, V_A = \dot{q}\, L_{Ax}\, L_{Ay}\, h, \tag{34}$$

and neither $\dot{q}$ nor V_A changes when sector A is relocated. The generated power is therefore invariant under any change of position.

At steady state nothing accumulates in the plate, since the storage term $\partial(\rho c\, T)/\partial t$ vanishes. Energy conservation then requires that all generated power leave through the boundaries at every instant,

$$Q_{\text{gen}} = Q_S + Q_W + Q_E. \tag{35}$$

Because the left-hand side is fixed at 8.0 W, the total dissipated power is likewise fixed, independently of where sector A sits. What the position of A controls is not the amount of power but its spatial routing: how the fixed total is divided among Q_S, Q_W and Q_E, and how high the temperature must rise inside the plate to drive that flux out through the available sinks. The two translations of Figure 6 are precisely a map of this routing, the symmetric one reflecting the two equal convective walls and the monotonic one reflecting the contrast between the adiabatic and the Dirichlet wall.

Finally, the quantity that is conserved and configuration-independent is a *power*, a rate in W, not a stored energy: the system generates 8.0 W and dissipates 8.0 W regardless of where sector A is placed. The thermal energy actually stored in the plate, $\int \rho c\, T \,\mathrm{d}V$, does differ between configurations, because the temperature fields differ, but it is constant in time and plays no part in the steady balance. The position of sector A governs not the magnitude of the energy exchanged but its distribution: the peak temperature reached and the share of the load carried by each surface. From an engineering standpoint this analysis shows us that in a component with fixed internal generation, the design can not reduce the total heat to be removed, but it can influence the maximum temperature, through the placement of the heat source relative to the available sinks. Locating the source near a sink such as the Dirichlet boundary minimises the peak; locating it against an adiabatic boundary maximises it.

4 Code

```
%-------------------------------------------------------------------------
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import RegularGridInterpolator
from time import time

t0 = time()
#########################################################################
#########################################################################
# Starting parameters
xLen = 0.1 # meters
yLen = 0.1 # meters
xLenA = 0.04 # meters
yLenA = 0.04 # meters
height = 0.01 # meters
kA = 100 # W/m/K
kB = 550 # W/m/K
qA = 5 * 10**5 # W/m3
TS = 30 # Celsius
TWE = 15 # Celsius
hb = 200 # W/m2/K

# Derived parameters
nx = 20 # number of volumes in the x direction
ny = 20 # number of volumes in the y direction
dx = xLen / nx # length of a single volume in x
dy = yLen / ny # length of a single volume in y
nVol = nx * ny # number of volumes

assert nx % 5 == 0, "The␣number␣of␣volumes␣in␣x␣must␣be␣a␣multiple␣of␣5"
assert ny % 5 == 0, "The␣number␣of␣volumes␣in␣y␣must␣be␣a␣multiple␣of␣5"

# Helpers
config = 1   # 1 for case 1 (top-left), 2 for case 2 (bottom right)
def inSectorA(i, j):
    nAx = round(xLenA/dx)
    nAy = round(yLenA/dy)
    if config == 1:
        return (i < nAx) and (j >= ny - nAy)
    else:
        return (i >= nx - nAx) and (j < nAy)

def cellK(i, j):
    return kA if inSectorA(i, j) else kB

def faceK(i, j, ni, nj):
    kP = cellK(i, j)
    kNB = cellK(ni, nj)
    return (2 * kP * kNB)/(kP + kNB)


# Gauss-Seidel parameters
iterMax = 30000
toll = 10**(-6)

#########################################################################
#########################################################################
# Finite volume discretization
xp = np.linspace(dx/2, xLen-dx/2, nx)  # x coordinates for centroids
yp = np.linspace(dy/2, yLen-dy/2, ny)  # y coordinates for centroids
```

```
# Building coefficients
a_w = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        if i != 0:
            a_w[j, i] = (faceK(i, j, i - 1, j) * dy * height)/dx


a_e = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        if i != nx - 1:
            a_e[j, i] = (faceK(i, j, i + 1, j) * dy * height)/dx


a_n = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        if j != ny - 1:
            a_n[j, i] = (faceK(i, j, i, j + 1) * dx * height)/dy


a_s = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        if j != 0:
            a_s[j, i] = (faceK(i, j, i, j - 1) * dx * height)/dy


b = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        if inSectorA(i, j):
            b[j, i] += qA * dx * dy * height


a_p = np.zeros((ny, nx))
for j in range(ny):
    for i in range(nx):
        a_p[j, i] = a_w[j, i] + a_e[j, i] + a_n[j, i] + a_s[j, i]


for i in range(nx):
    a_b = cellK(i, 0) * dx * height / (dy/2)
    a_p[0, i] += a_b
    b[0, i] += a_b * TS


for j in range(ny):
    a_rob = (dy * height)/((dx/(2 * cellK(0, j))) + (1/hb))
    condition (West face)
    a_p[j, 0] += a_rob
    b[j, 0] += a_rob * TWE


for j in range(ny):
    a_rob = (dy * height)/((dx/(2 * cellK(nx - 1, j))) + (1/hb))
    a_p[j, nx - 1] += a_rob
    b[j, nx - 1] += a_rob * TWE


# Scarborough criterion
```

```
neighbors = a_w + a_e + a_n + a_s
assert np.all(a_p > 0)
assert np.all(a_p >= neighbors - 1e-9)
assert np.any(a_p > neighbors + 1e-9)

# Padding
a_w = np.pad(a_w, 1)
a_e = np.pad(a_e, 1)
a_n = np.pad(a_n, 1)
a_s = np.pad(a_s, 1)
b = np.pad(b, 1)
a_p = np.pad(a_p, 1)

###############################################################################
###############################################################################
# GS setup
T = np.zeros((ny + 2, nx + 2))
iter = 0

# Gauss-Seidel
for k in range(0, iterMax):
    Told = np.copy(T)
    for j in range(1, ny + 1):
        for i in range(1, nx + 1):
            RHS = a_w[j, i] * T[j, i - 1] + a_e[j, i] * T[j, i + 1] +
            + a_n[j, i] * T[j + 1, i] + a_s[j , i] * T[j - 1, i] +
            + b[j, i]
            T[j, i] = RHS / a_p[j, i]
    iter += 1
    norm = np.linalg.norm(Told - T)
    if norm < toll:
        break


T_final = T[1:ny+1, 1:nx+1].copy()

t1 = time()
time = t1 - t0

###############################################################################
###############################################################################
# Validating the solution
R = np.zeros((ny+2, nx+2))
for j in range(1, ny+1):
    for i in range(1, nx+1):
        nb = (a_w[j, i] * T[j, i-1] + a_e[j, i] * T[j, i+1] +
        + a_n[j,i] * T [j+1, i] + a_s[j,i] * T[j-1, i] + b[j, i])
        R[j,i] = nb - a_p[j, i] * T[j, i]
assert np.abs(R).max() < 1e-4, "Discrete␣equation␣not␣resolved␣for␣all␣the␣cells"


for j in range(ny):
    for i in range(nx-1):
        assert abs(a_e[j + 1, i + 1] - a_w[j + 1, i + 2]) < 1e-9,
        f"E/W␣face␣mismatch␣at␣({i},{j})"

for j in range(ny-1):
    for i in range(nx):
        assert abs(a_n[j + 1, i + 1] - a_s[j + 2, i + 1]) < 1e-9,
        f"N/S␣face␣mismatch␣at␣({i},{j})"

###############################################################################
###############################################################################
```

```
# Energy calculation
qgen = qA * xLenA * yLenA * height  # Generated power

Tsouth = T_final[0, :]
Twest = T_final[: , 0]
Teast = T_final[:, nx - 1]

qs = 0
area = dx * height
for i in range(0, nx):
    qs += cellK(i, 0) * ( (Tsouth[i] - TS) / (dy/2) ) * area

qw = 0
area = dy * height
for j in range(0, ny):
    qw += ( (Twest[j] - TWE) / ( (dx/2) / cellK(0, j) +  1 / hb) ) * area

qe = 0
area = dy * height
for j in range(0, ny):
    qe += ( (Teast[j] - TWE) / ( (dx/2) / cellK(nx - 1, j) +  1 / hb) ) * area


assert (abs(qgen - (qs + qw + qe)) / abs(qgen)) < 1e-3, "Stationary condition not satisfied"

#########################################################################
#########################################################################
# Plotting contour figure
plt.figure(1)
X,Y = np.meshgrid(xp,yp)
plt.contourf(X,Y,T_final,30)
if config == 1:
    plt.title('Figure 1', fontsize=15)
else:
    plt.title('Figure 2', fontsize=15)
plt.xlabel('longitude x [m]')
plt.ylabel('longitude y [m]')
plt.xlim(0.0, xLen)
plt.ylim(0.0, yLen)
plt.grid(True)
plt.colorbar()
plt.set_cmap('jet')
plt.show()

# Plotting diagonal figure
interp = RegularGridInterpolator((yp, xp), T_final, bounds_error=False, fill_value=None)
N = 1000
t = np.linspace(0, 1, N)
x_line = 0.0  + t * (xLen - 0.0)
y_line = yLen + t * (0.0  - yLen)
pts = np.column_stack([y_line, x_line])
T_line = interp(pts)
s = np.hypot(x_line - x_line[0], y_line - y_line[0])
plt.figure(2)
plt.plot(s, T_line, '-', markersize=3)
plt.xlabel('distance along diagonal [m]')
plt.ylabel('temperature [ C ]')
plt.title('Temperature along the diagonal through sector A')
plt.grid(True)
plt.show()

# Print onscreen
Tmax = T_final.max()
```

```
Tmin = T_final.min()
if config == 1:
    print('\n------------------ First part (Figure 1) ----------------------')
else:
    print('\n------------------ Second part (Figure 2) ----------------------')
print('                    n. of volumes = {} '.format(nVol))
print('                    dx = {:1.4f} m'.format(dx))
print('                    dy = {:1.4f} m'.format(dy))
print('                    T max = {:1.2f} celsius'.format(Tmax))
print('                    T min = {:1.2f} celsius'.format(Tmin))
print('                    Qgen = {:1.2f} W'.format(qgen))
print('                    n. of iterations = {} '.format(iter))
print('                    Machine time = {:1.1f} sec'.format(time))

%--------------------------------------------------------------------------
```

5 Declaration of AI use

In accordance with the faculty guidelines on the use of artificial-intelligence tools in academic work, this section declares the use of such tools in the preparation of the present report.

A generative-AI assistant, Claude (Anthropic, model Opus 4.8) [4], was used as a technical aid during the development of the work. Its use was confined to a supporting role and did not replace the author's analysis, implementation, or conclusions. Specifically, the assistant was used for the following purposes:

- to clarify the geometric constraint that sector A imposes on the choice of control-volume dimensions;
- to review the algebraic derivations of the finite-volume discretization (face fluxes, harmonic-mean interface conductivity, Dirichlet, Robin and Neumann boundary treatments) and to check their internal consistency with the course notation and with the sketch of the solution provided by the user;
- to assist with the wording and structure of the report in English, and with the formatting of equations, tables and figures in LaTeX;
- to produce, from the problem specification, an independent solution whose results were compared against those of the author's own code, as a cross-validation of the numerical results;
- to adapt the original script for the analysis presented in Figure 6.

Every suggestion produced by the assistant was independently verified before being incorporated. The numerical results reported here were obtained exclusively from the author's own Python code; the algebraic derivations were re-derived and checked by hand against the course notes; and the physical arguments were confirmed against the computed temperature fields and energy balances. No text, equation, or result was accepted without this verification.

The artificial-intelligence tool is acknowledged solely as a technical aid and is not, in any sense, an author or co-author of this work. Full responsibility for the contents, interpretations, and conclusions of the report rests with the author.

Upon completion, the source code (`2D_diffusion.py`) and the present manuscript were submitted to a generative-AI assistant, Claude (Anthropic, model Fable 5) [5], for an independent review of their internal consistency.

References

[1] Serafín, E. A., Heidenreich, E., & Moreira, S. (n.d.). *Simulación numérica de fenómenos de transporte*.

[2] Quarteroni, A., Sacco, R., & Saleri, F. (2007). *Numerical mathematics* (2nd ed.). Springer.

[3] Gabrielli, G., Bucci, T., Cerri, G., & Guindani, B. (2023). *Real and functional analysis*. https://www.fubinitonelli.it/rfa/FubiniTonelli-RFANotes.pdf

[4] Anthropic PBC. (2026a, May 28). *Introducing Claude Opus 4.8*. https://www.anthropic.com/news/claude-opus-4-8

[5] Anthropic PBC. (2026b, June 9). *Claude Fable 5 and Claude Mythos 5*. https://www.anthropic.com/news/claude-fable-5-mythos-5